UNIEVANGÉLICA

RELATÓRIO - ORDENAÇÃO DINÂMICA COM MÚLTIPLOS ALGORITMOS

Vinicius Siqueira - Árvores e Grafos

Juliano Junior Barbosa Gonçalves[1] *,Guilherme Morais Ribeiro*[2] *, Eduardo Brasil Buhrer* [3] *, Murilo Magalhães Ribeiro*[4] *, Maria Luiza de Paula Rezende*[5] *, Samuel Nascimento*[6] *, Gabriel Ferreira Gomes*[7] *, Rafael Henrique Lima Cosmo*[8] *, Thiago Resende Gonçalves*[9] *, Luiz Henrique Gomes Laurentino*[10] *.*

2026

Relatório – Ordenação Dinâmica com Múltiplos Algoritmos

Vamos abordar no trabalho uma implementação em um dos algoritmos, com o objetivo de testar o desempenho desses 07 algoritmos: **Bubble Sort, Selection Sort, Insertion Sort**, **Quick Sort**, **Merge Sort**, **Shell Sort e Heap Sort**

Qual algoritmo teve melhor desempenho?

O que obteve o melhor desempenho foi o **Quick Sort**, **Merge Sort** e **Heap Sort**. Esses algoritmos eles tem a complexidade média de **O(n log n)**, que significa que conseguem ordenar grandes quantidades de dados de maneira mais eficiente do que algoritmos quadráticos. O **Quick Sort GERALMENTE** apresenta desempenho muito bom na prática, sendo bastante utilizado em implementações reais de ordenação.

Qual teve pior desempenho?

Os algoritmos Bubble Sort e Selection Sort teve o pior desempenho. Os dois possuem complexidade $O(n^2)$, que significa que o número de comparações cresce rapidamente conforme o tamanho do vetor aumenta. Por isso, esses algoritmos são considerados menos eficientes para grandes conjuntos de dados. Ele é mais utilizado apenas para fins educacionais ou para vetores muito pequenos.

Qual parece mais adequado para inserção dinâmica?

Foi o **Insertion Sort**. Porque ele trabalha inserindo cada novo elemento na posição correta dentro de uma parte do vetor que já está ordenada. Quando esses novos elementos são adicionados, ele vai precisar realizar só pequenos ajustes na estrutura, tornando a execução mais eficiente nesse tipo de cenário. O que acontece quando o vetor já está quase ordenado? Quando o vetor já está quase ordenado, alguns algoritmos apresentam melhor desempenho. O **Insertion Sort**, pode alcançar desempenho próximo de **O(n)** nesse caso. No outro lado, os algoritmos como **Bubble Sort** e **Selection Sort** continuam realizando muitas comparações mesmo quando os dados já estão praticamente ordenados, o que tira a eficiência. Os algoritmos mais avançados como **Quick Sort**, **Merge Sort** e **Heap Sort** vão continuar mantendo desempenho estável próximo de **O(n log n)**.

Complexidade teórica dos algoritmos

Algoritmo	Melhor Caso	Caso Médio	Pior Caso
Bubble Sort	O(n)	$O(n^2)$	$O(n^2)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$
Insertion Sort	O(n)	$O(n^2)$	$O(n^2)$
Quick Sort	O(n log n)	O(n log n)	$O(n^2)$
Merge Sort	O(n log n)	O(n log n)	O(n log n)
Shell Sort	~O(n log n)	~O(n log n)	$O(n^2)$
Heap Sort	O(n log n)	O(n log n)	O(n log n)

Conclusão

A análise mostrou pra gente, que os algoritmos de complexidade **O(n log n)** são mais eficientes pra maior volume de dados. Mas, em cenários de inserção dinâmica, algoritmos como **Insertion Sort**, pode apresentar bom desempenho, especialmente quando o vetor já está parcialmente ordenado. Então, a escolha do algoritmo ideal, vai depender do contexto de uso e das características dos dados a serem ordenados.